

FRED TALKS

2nd July 2026

CONTENTS

Please for the love of all that is holy: Stop writing long boring
Titles

SWYX · 1431 WORDS

Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 2207 WORDS

[Daily article] July 2: No. 1 Aircraft Depot RAAF

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 398 WORDS

Holes

XKCD.COM · 24 WORDS

We're coming out of stealth

ETCHED · 43 WORDS

Voice In, Visuals Out @ AI Engineering World's Fair

ALLEN PIKE · 41 WORDS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · [SOURCE](#)

People code switch. Every ethnic minority has [lived experience of this](#). When you talk to me in Singapore, I sound very different than when I’m in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I’ve been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it’s all “***Leveraging GenAI with Robust Data Foundations to Transform Businesses***”. I’ve left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don’t have good title game. That’s ok. You’re great at what you do, title game isn’t your job. I’m the guy that wrote [How To Name Things](#) and [Two Words](#). I sent you this guide *because* your shitty title is the main thing stopping your readers from getting to the good part. If you didn’t have anything good, I wouldn’t bother with a lost cause.

First...

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with “Leveraging” or “Towards” or “Navigating” or “Empowering”

Yeah just stop. There’s no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

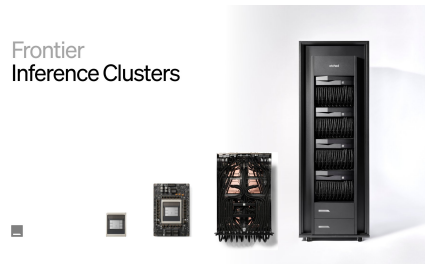
Make It Quotable

If you do not try to make your title roll off the tongue, it probably won’t.

We've built our first racks after a successful A0 tapeout, \$1B+ in customer contracts, and \$800m raised.

Early customer tests show us achieving SOTA throughput, latency, and power efficiency on inference workloads.

Our first racks ship this summer.



Voice In, Visuals Out @ AI Engineering World's Fair

ALLEN PIKE · 29 JUN 2026 · [SOURCE](#)

This week's AI Engineering World's Fair just posted [my talk on the agony and ecstasy of voice in, visuals out agents](#). It's a challenge to get model responses that feel immediate, but when it works, it feels magical.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. [Simple Made Easy](#). [Inventing on Principle](#). Our list of best [demos/pitches](#) of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not [High School Musical: The Musical: The Series](#).

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule:** If you repeat yourself [three times](#), ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My [Radiating Circles](#) and [Measuring Developer Relations](#) and [Third Age of JavaScript](#) did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. [What Clayton Christensen Got Wrong](#) made Ben Thompson's career. [Rise of AI Engineer](#) countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.

Curiosity Gap Dynamics

Marketers know the power of the Curiosity Gap. Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to “burying the lede”, which make your talk less Quotable and you also don’t want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
 - e.g. How to Get Rich (Without Getting Lucky), How to Market Yourself (Without Being A Celebrity). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
 - e.g. Parse, don't validate
 - alternative prescriptive opinion: **Never** {*THING YOU DO*}
 - see also title of this post
- {*POPULAR THING*} **IS DEAD**
 - unfortunately this is clickbait that works
 - e.g. Prompt Engineering is Dead, Big Data is Dead
 - when given at a conference for *POPULAR THING*, this is known as a Heresy Talk

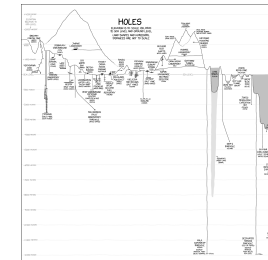
relentlessly: 1. In a relentless manner. 2. In a manner unrelenting or unyielding in severity; unrelentingly. 3. In a steady and persistent manner; unceasingly. 4. About Word of the Day 5. Nominate a word 6. Leave feedback <https://en.wiktionary.org/wiki/relentlessly>

Wikiquote quote of the day:

These people were worthy of love and admiration in their blind loyalty, in their blind strength and tenacity. With the exception of one small thing, one tiny little thing, they lacked nothing that the sage and thinker had, and that was the consciousness of the unity of all life. ~ --Hermann Hesse https://en.wikiquote.org/wiki/Hermann_Hesse

Holes

XKCD.COM · 02 JUL 2026 · [SOURCE](#)



If you're thinking 'Wait, a giant crystal cave in Mexico? What's that?' then I'm SO excited for the image search you're about to do.

We're coming out of stealth

ETCHED · 30 JUN 2026 · SOURCE

We're coming out of stealth.

No. 1 Aircraft Depot (No. 1 AD) was a maintenance unit of the Royal Australian Air Force (RAAF). Formed in July 1921 at RAAF Point Cook, Victoria, it relocated to RAAF Laverton in March 1926. In its early years, the depot serviced aircraft and other equipment, trained maintenance staff, and supported survey flights in Australia and the Pacific region. No. 1 AD's strength increased from 350 staff in the 1930s to more than 2,000 during World War II, when it assembled, tested and repaired aircraft such as Tiger Moth trainers, Spitfire fighters, and B-17 Flying Fortress heavy bombers. After World War II, No. 1 AD introduced the first jets into RAAF service. In 1961, it ceased airframe maintenance but continued to service aero engines. By the 1970s, the depot's focus was ground-based equipment, though it still handled some aircraft components. No. 1 AD was disbanded in December 1994; at the time of its disbandment, it was the oldest RAAF unit in continuous operation. (Full article...).

Read more: https://en.wikipedia.org/wiki/No._1_Aircraft_Depot_RAAF

Today's selected anniversaries:

1816:

The French frigate Méduse ran aground off the coast of present-day Mauritania, with the survivors escaping on a makeshift raft, depicted in Théodore Géricault's painting The Raft of the Medusa. https://en.wikipedia.org/wiki/The_Raft_of_the_Medusa

1941:

A German SS unit arrived in Vilnius, Lithuania, and began the systematic execution of up to 100,000 people over the next three years. https://en.wikipedia.org/wiki/Ponary_massacre

1976:

More than a year after the end of the Vietnam War, North and South Vietnam officially merged under communist rule to form the Socialist Republic of Vietnam. <https://en.wikipedia.org/wiki/Vietnam>

2013:

The International Astronomical Union announced that the fourth and fifth moons of Pluto to be discovered would be named Kerberos and Styx, respectively. https://en.wikipedia.org/wiki/Styx_%28moon%29

Wiktionary's word of the day:

◦ opposite also works:

▪ **The Rise of {THING}**

▪ **The Unreasonable Effectiveness of {THING}**

• **Fundamentals of {THING}**

◦ straightforward, people love fundamentals

◦ also **How {THING} Really Works**

◦ You can also flip it and go for the **Advanced {THING}** tier content

• {CONVEY EXPERIENCE/SCALE} with {THING}

◦ e.g. What We Learned from a Year of Building with LLMs, Insights from over **10,000** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you’ve got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I’m currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.
- My next iteration was “**Stop writing model routers, use an LLM Gateway**”. Ok, prescriptive, evocative. But didn’t cover the other elements other than model routing. One could shorten to “Stop writing your own LLM Gateway” but that doesn’t quite express what people do.
- My next one is “**It’s not your job to DIY an LLM Gateway**”. No buzzword bc “your job” is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.
- Final one is “**LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions**”. Leans on Tim Ferriss’ madlbs.

It’s very common for the “tail to wag the dog to wag the tail” - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

- Save or copy rendered images from a task.
- See queued prompts while Codex is still running.
- Open completed tasks directly from notifications.
- Browse archived threads instead of keeping the active list crowded.
- Wrap long diff lines when reading on a narrow screen.
- Use Face ID or the device passcode to protect Codex separately.
- See skills and plugins inline in the composer before sending.
- Capture a photo directly into a prompt when the real-world object is the context.
- Start a side chat from selected transcript text rather than re-explaining the question.

None of these changes the fundamental agent model. Together, they remove the friction that makes a remote workflow feel fragile.

The larger lesson

The best mobile software does not shrink a desktop interface. It identifies the decisions that matter when you are away from the desk and makes those decisions fast, legible, and safe.

That is how I think about Codex Remote now. It is where I can choose the right environment, set the objective, redirect a run, answer an approval, inspect the result, and keep the whole queue of engineering work coherent.

The computer still does the work. The phone keeps me in control.

[Daily article] July 2: No. 1 Aircraft Depot RAAF

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 02 JUL 2026 · [SOURCE](#)

10. Five workflows that work especially well on mobile

The release captain

Start a focused thread for a release or pull request. Ask Codex to check the current branch, CI state, open review feedback, and release inclusion. Pin the thread. When new information arrives, steer only when it invalidates the active investigation; otherwise, queue it. Review the final diff or release note from the phone and archive the thread when the release lands.

The interrupt-driven bug fix

Attach a screenshot, log, or captured file. Ask Codex to diagnose before editing. Use a side chat to interrogate one suspicious error without derailing the main investigation. Once the cause is clear, return to the main thread and authorize the narrow fix.

The mobile reviewer

Run a review against the intended branch, inspect the changed-file summary, open the files that matter, and attach inline comments. Ask Codex to address only those comments, then review the follow-up diff.

The long-running objective

Create a goal with a concrete completion condition: Tests green, review feedback resolved, or a reproducible performance threshold met. Check progress from notifications and status rather than repeatedly asking “Are you done?” Use queued prompts for additional work and reserve steering for corrections.

The multi-machine operator

Keep hosts named clearly and organize work by machine and workspace. Use the phone to start a task on the machine that has the right checkout, credentials, simulator, or operating system. This is especially useful when one task requires a Mac and another needs a Windows host.

Small features with outsized impact

Some of my favorite additions are not headline features:

- Edit the latest sent prompt instead of adding a correction turn.

Subscribe to our newsletter

Mastering Codex Remote for engineering

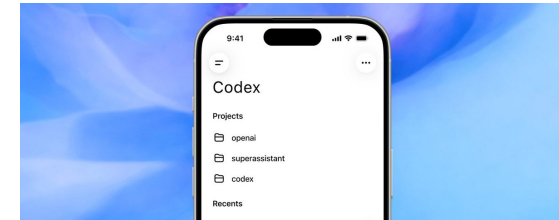
DEVELOPERS.OPENAI.COM · 25 JUN 2026 · [SOURCE](#)

Codex Remote is easy to underestimate.

At first glance, it looks like a way to check on a coding task from your phone. That is useful, but it misses the bigger idea. The real power of Codex Remote is that it lets you start, direct, review, and organize work running on your development machines without pretending that an iPhone should be a tiny terminal.

Over the last two months, we have added a surprising amount of depth to that loop: remote host connections, worktrees, goals, side chats, inline code review, queued and steering prompts, attachments, skills and plugins, archived threads, security controls, and a long tail of small details that make the app useful for serious work.

This is the field guide I wish every new power user had.



The right mental model: Your phone is the control plane

The code still runs where it belongs: on your Mac, Windows machine, devbox, or other connected host. Codex Remote gives you a native interface for controlling that work.

That distinction matters. The goal is not to reproduce every terminal affordance on a small screen. The goal is to make the decisions that unblock an agent easy from anywhere:

- What repository and workspace should it use?

- Should this run in the current branch or a fresh worktree?
- Should my next message wait, or should it redirect the active turn?
- Is this command safe to approve?
- What changed, and do I agree with it?
- Should this become a durable goal, a separate thread, or a quick side question?

Once you use the app this way, it stops feeling like remote desktop software and starts feeling like an engineering control plane.

1. Start the task with the right boundaries

Good agent work starts with a correctly scoped environment. Codex Remote lets you choose the connected host and workspace before the first prompt. For a new thread, you can also choose a branch, create a separate worktree, and run the environment setup associated with it.

That makes a few useful patterns possible:

- Use the current checkout for a quick investigation.
- Create a new worktree for a change that should stay isolated.
- Start from the intended base branch instead of fixing Git state later.
- Let the environment setup run before asking Codex to build or test.

This is one of the most important power-user habits: Spend 10 seconds choosing the right execution context and save 10 minutes of cleanup later.

The composer can also carry more context than plain text. You can attach files, photos, or a fresh camera capture. Skills and plugins appear inline, which makes it much easier to confirm that your prompt is invoking the capability you intended.

My rule is simple: If a screenshot, file, or named skill can remove ambiguity, attach or mention it before the first turn.

I pin the few threads that are actively moving, rename them around outcomes, and archive them aggressively once the work is done. Archived-thread browsing makes this safe: Archive is organization, not deletion.

Notifications are part of that system. A completion notification can open the relevant Codex task directly, so the handoff from “agent finished” to “human reviews” is one tap.

Spotlight and Shortcuts can open Codex Remote directly. On iPad, keyboard shortcuts make the app surprisingly efficient: Create a new thread, move between threads, open changed files, pin, rename, and archive without reaching for every control.

This is the “chief of staff” use case for Codex Remote. The app is not only where I issue prompts. It is where I keep track of which pieces of engineering work are active, blocked, waiting for review, or finished.

9. The hidden command palette

Typing / reveals the fastest route to many of the app’s advanced behaviors. Availability can depend on the connected host, app version, and account configuration.

Command	What it is for
/plan	Toggle Plan mode before implementation.
/goal <objective>	Create or update a durable objective.
/side [question]	Ask a side question without disrupting the main thread.
/review	Review local changes or compare them with a branch.
/status	Inspect the session, workspace, context, and rate limits.
/compact	Compress a long thread’s context.
/fork	Create a new primary thread from the current history.
/fast	Switch between standard and faster execution when available.
/feedback	Send product feedback tied to the current session.

The command palette is worth learning because it exposes the product’s real conceptual model: Plan, pursue, branch, review, inspect, and recover.

Codex Remote surfaces approval requests for commands, file changes, network access, and connected tools. Depending on the request and host configuration, an approval may apply once, for the current chat, or more broadly.

The power-user move is not to approve everything. It is to choose the narrowest permission that keeps the work moving.

For a well-understood command in a trusted thread, a chat-scoped approval can remove repetitive interruptions. For an unfamiliar command, a sensitive repository, or a request whose effect is unclear, approve once—or deny it and ask Codex to explain or use a safer approach.

You can also choose the thread’s broader approval behavior when starting work. Think of this as part of task setup, alongside the host, workspace, branch, and model.

7. Manage context before it becomes a problem

Agent threads are stateful, and long threads eventually accumulate enough context to become slower or less focused.

Codex Remote has a small set of tools for managing that lifecycle:

- `/status` shows session details, workspace, context usage, and available rate-limit information.
- The optional context indicator keeps remaining context visible in the composer.
- `/compact` compresses an overgrown thread while preserving the useful working state.
- `/fork` creates a new thread from the current one when you want shared history but a different direction.

The practical sequence is: Check status, compact when the objective is still the same, and fork when the objective has diverged.

Do not use a side chat and a fork interchangeably. A side chat is a lightweight question around the current work. A fork is a new primary line of work that inherits the thread’s history.

8. Keep a clean Thread Desk

My own Codex workflow increasingly looks like a small operations desk.

2. Learn the difference between Queue and Steer

This is probably the least obvious high-leverage setting in Codex Remote.

When Codex is already working, a follow-up can do one of two things:

- **Queue** waits until the current response finishes, then sends your prompt as the next turn.
- **Steer** injects guidance into the work already in progress.

Queue is the safe default. Use it for a second task, an extra test request, or anything that should happen after the current work.

Steer is for correcting direction while the cost of continuing down the wrong path is still growing:

- | Keep the fix inside the mobile package. Do not refactor the shared renderer.
- | The reproduction only happens after reconnect. Test the resumed path, not the live path.
- | Stop investigating the UI. Check whether the server removed the item during resume.

That turns the phone into something more useful than a status screen. You can intervene at the exact moment a run needs judgment.

You can choose the default follow-up behavior in Codex settings. I keep Queue as the default and use Steer deliberately; accidental mid-turn redirection is usually more expensive than waiting.

3. Use side chats as a branch of thought

Long-running coding threads accumulate valuable context. Interrupting one with every side question makes the main transcript noisy and can pull the agent away from the objective.

That is what side chats solve.

Use `/side` to open a lightweight conversation connected to the current thread. Use `/side <prompt>` to open it with a question already prepared. Even better, select text in the transcript and choose **Ask in side chat**. The selected passage becomes the starting context for the new conversation.

I use side chats for questions such as:

- Why did Codex choose this architecture?
- What does this error actually mean?
- Is this behavior consistent with the desktop app?
- Give me a release-note version of this implementation detail.
- What should I verify before I approve this command?

The distinction is useful: The main thread owns the work; the side chat helps me understand the work.

4. Use Plan for the path and Goal for the outcome

Plan mode and goals solve different problems.

Plan mode asks Codex to propose the implementation path before changing code. It is useful when the task is underspecified, risky, or likely to touch several systems.

A goal is durable. It tells Codex what outcome to keep pursuing across turns. On mobile, `/goal` can create and manage that objective, while progress remains visible as the work continues.

A practical pattern is:

1. Start with Plan mode for a risky change.
2. Inspect the proposed boundaries.
3. Turn the accepted outcome into a goal when the work will require multiple iterations.
4. Let Codex continue through implementation, tests, review feedback, and cleanup without restating the objective every time.

Plans answer “How should we approach this?” Goals answer “What must be true before we are done?”

5. Review code without leaving the conversation

The review loop is where Codex Remote becomes genuinely useful for engineering rather than merely convenient.

Completed turns can show a summary of changed files. From there, you can open the diff, inspect individual files, expand or collapse sections, wrap long lines, and open a source file with syntax highlighting. You can attach inline comments to the relevant lines and send that review context back to Codex.

There are several levels to the workflow:

- Open the changed-file summary for a quick sanity check.
- Tap through to the full source when a diff lacks surrounding context.
- Add inline comments for precise corrections.
- Use the review command to review local changes or compare against a branch.
- Link a file back into the chat when you want Codex to reason about that file specifically.

This makes a tight mobile review loop possible:

1. Codex completes the implementation.
2. I inspect the diff from my phone.
3. I leave two inline comments.
4. Codex addresses them in the same thread.
5. I review the smaller follow-up diff.

The important point is not that a phone replaces a large monitor for deep code reading. It does not. The point is that many reviews are blocked on one or two decisions, and those decisions no longer need to wait until I am back at my desk.

6. Treat permissions as part of the workflow

Remote work is only useful if control stays explicit.